🏗️ Modern Frontend Architecture: A Summary

Speaker: Tejas Kumar

Core Theme: Moving beyond the "SPA vs. SSR" debate to focus on intentional architecture.

1. The "Why" of Architecture

Tejas argues that architecture isn't about using the "coolest" framework (React, Vue, Svelte, etc.). Instead, it is the process of mapping **Business Requirements** to **Technical Implementations**.

- **The Problem:** Many developers pick a stack first and then try to fit the problem into it.
- **The Solution:** Understand the constraints of your users (network speed, device power) before writing a single line of code.

2. The Evolution of Rendering Patterns

The talk breaks down the three primary ways we deliver content to users today. Choosing the right one is the foundation of modern architecture:

Pattern	How it Works	Best For...
CSR (Client-Side Rendering)	The browser downloads a blank HTML file and a large JS bundle to build the page.	Highly interactive dashboards, logged-in states.
SSR (Server-Side Rendering)	The server generates the HTML on every request.	E-commerce, content that changes frequently but needs SEO.
Static Generation	HTML is generated at build time and served via CDN.	Documentation, blogs, marketing sites.

3. The Concept of "Islands" and Partial Hydration

Tejas highlights a shift in modern architecture (seen in frameworks like **Astro** or **Fresh**) where we stop treating the whole page as a single JavaScript application.

- **Hydration is Expensive:** Sending 500KB of JS just to make a "Submit" button work is poor architecture.
- **The "Islands" Approach:** Send static HTML for the layout (header, text, footer) and only "hydrate" (add JS to) the specific interactive parts of the page.

4. Data Fetching Strategies

A major part of the talk focuses on where data should live.

- **Colocation:** Keeping your data fetching logic close to the component that needs it.
- **Type Safety:** Using tools like **TypeScript** and **Zod** to ensure that what the API sends is actually what the frontend expects, reducing runtime crashes.

💡 **Key Takeaways for Developers**

- **Don't over-engineer:** If a static HTML page solves the problem, don't build a complex React SPA.
- **Performance is a feature:** High LCP (Largest Contentful Paint) and FID (First Input Delay) scores are architectural failures, not just "CSS issues."
- **Evaluate your "Bundles":** Be conscious of how much JavaScript you are forcing the user to download.